

Design and Verification of RISC-V (RV32IM) Pipelined Processor

*submitted in partial fulfillment of the requirements
for the degree of*

BACHELOR OF TECHNOLOGY

in

ELECTRICAL ENGINEERING

by

GURRAM VENKATA TARUN KUMAR EE20B012

Supervisor(s)

Dr. VikramKumar Pudi

**DEPARTMENT OF ELECTRICAL ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY TIRUPATI**

May 2024

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/-data/fact/source in my submission to the best of my knowledge. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Place: Tirupati
Date: 07-05-2024

Signature
Gurram Venkata Tarun Kumar
EE20B012

Note: If more than one authors mentioned in the cover page, modify each 'I' by 'we' and then include remaining author's name at the bottom of the page. Signature of each author in this page is must.

BONA FIDE CERTIFICATE

This is to certify that the report titled **Design and Verification of RISC-V (RV32IM) Pipelined Processor** submitted by **Gurram Venkata Tarun Kumar**, to the Indian Institute of Technology, Tirupati, for the award of the degree of **Bachelor of Technology**, is a bona fide record of the project work done by them under my supervision. The contents of this report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Place: Tirupati
Date: 07-05-2024

Signature
Dr .VikramKumar Pudi
Assistant Professor
Department of Electrical
Engineering
IIT Tirupati - 517619

ACKNOWLEDGMENTS

We would like to express our special thanks of gratitude to Dr. Vikramkumar Pudi for giving his valuable suggestions during the planning and development of this work. His guidance was extremely helpful and motivating.

I would like to thank Dr Jaynarayan T Tudu, Assistant Professor Indian Institute of Technology. Tirupati, in guiding me through some key insights. I would like to thank the VLSI lab JTS Mr. Kumar for his help .Lastly, I would also like to thank our parents and friends, who were supportive throughout the project.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	i
LIST OF FIGURES	v
LIST OF TABLES	vii
ABBREVIATIONS	viii
NOTATION	ix
1 INTRODUCTION	1
1.1 Objectives	3
2 LITERATURE REVEIW	4
2.1 RV32I ISA	4
3 PROCESSOR DESIGN	7
3.1 Basic Functionality Modules	7
3.2 Single Cycle Architecture	10
3.3 Adding M extension to RV32I	16
3.4 Pipelining Design OF RV32IM	20
3.4.1 Designing	20
3.4.2 Hazards	21
4 VERIFICATION METHODOLOGY	23
4.1 Softwares used	23
4.2 Simulation Based Verification	23
4.2.1 Arithmetic operation programe	24
4.2.2 Fibonacci program	25
4.3 FPGA based verification	26
5 RESULTS AND DISCUSSION	28

6	SUMMARY AND CONCLUSION	29
A	Processor ISA Types	31
A.0.1	Background Intro	31
B	Data Path Modules Description and functionalities	33
C	Code snippets	37

LIST OF FIGURES

2.1	Instruction formats r1	5
2.2	RV32I Instruction Types r1	5
3.4	Result Write Back Mux	9
3.5	RV32I Data Path	10
3.6	R type Instruction data Path	11
3.7	L type Instruction data Path	11
3.8	RI type Instruction data Path	12
3.9	S type Instruction data Path	12
3.10	LUI type Instruction data Path	13
3.11	BR type Instruction data Path	13
3.12	JAL type Instruction data Path	14
3.13	JARL type Instruction data Path	14
3.14	AUIPC type Instruction data Path	15
3.15	M extension Computation Module	17
3.16	ALU Module for RV32IM	17
3.17	M extension Control Module	18
3.18	rv32im data path	19
3.19	Pipelined Microarchitecture Design	20
3.20	Data Hazard Handling	21
3.21	Control Hazard Handling	22
4.1	Data Hazards	24
4.2	Control Hazards	24
4.3	Instructions Causing Control Hazard	24
4.4	Instructions Causing Data Hazards	24
4.5	Arthamatic operations c++ code	25
4.6	Arthamatic operations output simulation	25
4.7	25th Fibanachi number calculation	26

4.8	FPGA Verification Flow	27
-----	----------------------------------	----

LIST OF TABLES

1.1	Table: Description of RISC-V Base and Extension Types	2
2.1	RV32I Instruction Purpose	5
2.2	Arithmetic Instructions	6
2.3	Logical Instructions	6
2.4	Comparison Instructions	6
2.5	Special Computation Instructions	6
2.6	Memory Access Instructions	6
2.7	Conditional Control Flow Instructions	6
2.8	Unconditional Control Flow Instructions	6
3.1	RV32M Standard Extension Instructions	17
3.2	Multiplication Instructions operation	18
3.3	Division Instructions Operation	18
3.4	Multiplication Instructions Control signals	18
3.5	Division Instructions Control signals	18
5.1	Delay results of RV32IM single cycle vs Piplined Designs	28
5.2	FPGA resource utilization	28
B.1	Instruction Memory Ports Description	33
B.2	Instruction Memory Asynchronous Reading for a single cycle	33
B.3	Instruction Memory Synchronous Reading for Multi-cycle or Pipelined Design	33
B.4	Instruction Memory Synchronous Writing	33
B.5	Data Memory Ports	34
B.6	Data Memory data to be stored control signals	34
B.7	Data Memory Synchronous Writing	34
B.8	DataMemory reads operation control signals	34
B.9	Data Memory Synchronous Reading	35

B.10 GPR's synchronous write & asynchronous read	35
B.11 ImmOp and Operation	35
B.12 Immediate Generator Functionality	35
B.13 ALU unit control signals	35
B.14 write back :rsltMux control signals	36

ABBREVIATIONS

ISA	Instuction set architecture
RISC-V	fifthe verion ISA developed based on Reduced Instruction Set Architecture principles
FPGA	Feild Programable Gate Array
UART	Universal asynchronous Receving and Transmison

NOTATION

rd	Destination register address of size 5 bits to store the computed value
rs1,rs2	Data address of size 5 bits of the data needed to be computed
Imm	32 bit Immediate value decode from the INstruction

CHAPTER 1

INTRODUCTION

RISC(Reduced Instruction Set Computer) offers advantages over CISC (Complex Instruction Set Computer) through simpler instructions, streamlined execution, and improved performance, enabling faster clock speeds and more efficient use of resources. RISC-V (Reduced Instruction Set Computer - Five) Instruction Set Architecture, a 5th major open-source RISC-based Instruction Set Architecture, originated from the efforts of researchers at the University of California, Berkeley, in 2010.

Unlike proprietary ISAs such as ARM or x86, RISC-V is freely available for anyone to use, modify, and distribute. It's modular design allows for flexibility in implementation, enabling developers to customize processors for specific applications or performance requirements. The architecture is designed to be extensible, allowing developers to add custom instructions or extensions to meet specific application requirements. It is a simple load and store architecture that supports 32-bit and 64-bit base integer instruction formats, with optional extensions for specific use cases, such as floating-point arithmetic, atomic operations, and vector processing which gives flexibility in terms of designing application-specific processors

RISC-V stands for "**Reduced Instruction Set Computer - Five**," where the "Five" refers to the fifth version RISC ISA designed by designers from the University of California, Berkeley. follows a **load-store architecture**, which is a type of Reg-Reg/Load-Store ISA. It has a relatively simple instruction set

Base	about	Status
RV32I	32 bit Integer GPR's	standard
RV64I	64 bit Integer GPR's	Ratified
RV32E	only 16 GPR's,used for embedded applications	Draft
RV128I	128 bit Integer GPR's	Draft
Extension	about	Status
M	Includes Direct Multiplication, Division ability	Ratified
A	Memory synchronization, Multi thread processing	Ratified
F	supports single precision Floating point operations	Ratified
Zicsr	enable software, to manage processor behaviour via CSR's	Ratified
C	supports compressed instructions	Ratified

Table 1.1: Table: Description of RISC-V Base and Extension Types

1.1 Objectives

The objective of this work is to design the RISC-V-based pipelined processor with a 32bit base ISA of RV32I along with M extension for direct multiplication and Division operations supportability and Its verification using different assembly programs ,along with the help of RISC-V GNU Toolchain for for converting the c and c++ programs into the required instructions to run and test on the Designed processor

CHAPTER 2

LITERATURE REVIEW

For the Design purpose, I have referred to the Computer Architecture RISC-V edition [Harris](#) book for designing principles of single cycle and pipeline processors. I have referred the [PICORV32_Processor](#), and [NEO32_Processor](#) for software firmware to use the gnu toolchain for getting the instructions to run on the Designed processor.

2.1 RV32I ISA

Any RISC-V isa-based instruction would be in the six formats shown in [Figure 2.1](#). The major are R, I, S, and U, while B and J are the same as S and U except for the immediate encoding.

There are 46 instructions in the RV32 base ISA in Specifications [Waterman \(2017\)](#) in 2017 version. later, they separated 6 instructions from it as an extension of Zicsr. According to the new version Specification [Waterman and Asanovic \(2019\)](#), there are 40 base instructions in the RV32I base ISA, whereas 2 are system instructions that play an important role in environment executions.

There are 9 types of instructions in 32-bit base ISA, The purpose of each type of Instruction is shown in the [Table 2.1](#) and their respective encoding formats are mentioned in [Figure 2.2](#)

There are 38 Instructions of our interest . [Tables 2.2, 2.3 2.4 and 2.5](#) show the computation instructions. Where the [Table 2.6](#) shows the load and store instructions. [Tables 2.7 and 2.8](#) show the conditional and unconditional jumps, respectively.

Format	7'[31:25]	5'[24:20]	5'[19:15]	3'[14:12]	5'[11:7]	7'[6:0]
R	func7	rs2	rs1	func3	rd	opcode
I	imm		rs1	func3	rd	opcode
S	imm[11:5]	rs2	rs1	func3	imm[4:0]	opcode
B	imm[12][10:5]	rs2	rs1	func3	imm[4:1][11]	opcode
J	imm[20][10:1][11][19:12]				rd	opcode
U	imm[20 bits]				rd	opcode

Figure 2.1: Instruction formats [r1](#)

Type	Purpose
R	Only Registers data computation
RI	Computation with constants
LD	Memory load
SR	Memory store
BR	Conditional Looped computations
JAL ,JALR	Calling routines ,subroutines
AUIPC	To jump to functions far from the current one
LUI	Loading larger constant values

Table 2.1: RV32I Instruction Purpose

Type	Format	7'[31:25]	5'[24:20]	5'[19:15]	3'[14:12]	5'[11:7]	7'[6:0] opcode
R	R	func7	rs2	rs1	func3	rd	0110011
RI	I	imm[11:0]		rs1	func3	rd	0010011
LD	I	imm[11:0]		rs1	func3	rd	0000011
SR	S	imm[11:5]	rs2 (Data)	rs1	func3	imm[4:0]	0100011
BR	B	imm[12][10:5]	rs2	rs1	func3	imm[4:1][11]	1100011
JAL	J	imm[20][10:1][11][19:12]				rd	1101111
JALR	I	imm[11:0]		rs1	func3	rd	1100111
AUIPC	U	imm[31:12]				rd	0010111
LUI	U	imm[31:12]				rd	0110111

Figure 2.2: RV32I Instruction Types [r1](#)

Instruction	Computation
ADD, ADDI	Addition
SUB	Subtraction
SLL, SLU	Shift logical data left
SRL, SRU	Shift logical data right
SRA, SRAI	Shift Arithmetic data right

Table 2.2: Arithmetic Instructions

Instruction	Computation
AND, ANDI	Bit-wise AND operation
OR, ORI	Bit-wise OR operation
XOR, XORI	Bit-wise XOR operation (also used for 1's complement computation)

Table 2.3: Logical Instructions

Instruction	Computation
SLT, SLTI	Set 1, if signed comparisons less than
SLTU, SLTIU	Set 1, if unsigned comparisons less than

Table 2.4: Comparison Instructions

Instruction	Computation
LUI	Load upper 20-bit immediate
AUIPC	Add upper 20-bit immediate to PC (jump to any address in the address space)

Table 2.5: Special Computation Instructions

Instruction	Computation
LB, SB	Load/store byte
LH, SH	Load/store half-word
LW, SW	Load/store word
LBU, LHU	Load/store unsigned byte, half-word

Table 2.6: Memory Access Instructions

Instruction	Operation
BEQ, BNE	Jump if (a=b), (a!=b) respectively
BGE, BLT	Jump if signed (a>=b), (a<b) respectively
BGEU, BLTU	Jump if unsigned (a>=b), (a<b) respectively

Table 2.7: Conditional Control Flow Instructions

Instruction	Operation
JAL	Jump to PC+imm (typically used for function calls)
JALR	jump to (rs1)+(imm) (typically used for function returns)

Table 2.8: Unconditional Control Flow Instructions

CHAPTER 3

PROCESSOR DESIGN

3.1 Basic Functionality Modules

Program Counter: The first mode of communication with the processor is instruction, which we can get from the Instruction memory, with the proper address, which is obtained from the program counter. The program counter holds the current address of the instruction, and its value is decided by the Mux, which has inputs of incremented PC value or branch or jump address value.

Immediate Generator The module shown in the figure 3.2b converts the encoded immediate value in the instruction to the required 32-bit Value according to the respective instruction format shown in the 2.1. **ALU 2nd input mux(aluIn2_Mux)"** We need to choose the data to be sent to the ALU unit using a mux. It chooses between the rs2data read from the GPRs or the immediate value decoded from the instruction; based on the control signal, it chooses one.

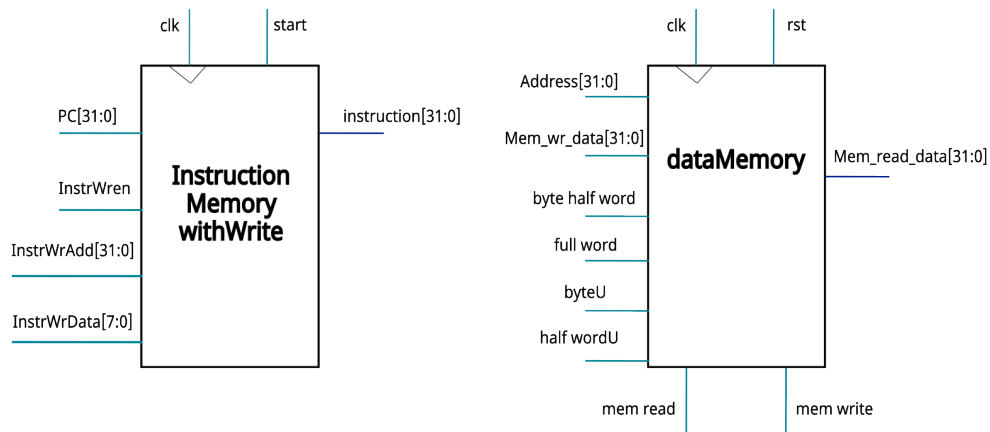
Data Memory: The Data memory shown in the figure 3.1b hold the required data must support the read and write operations when appropriate signals are enabled. We can use the asynchronous reading data memory in a single-cycle processor design with less data memory, which will be converted into the LUTRAMs in the FPGA when synthesized. When our data memory size is larger, we need to use the BRAM in the FPGA, making the memory synchronous read and synchronous write.

General Purpose registers(GPR's): Its dual port 32bit 32 number of registers stores the temporal data of the data memory. The first register is always zero. To write this module, we need a Demux and a Mux to access the register's data. The ports for this are shown in the the figure 3.3a

Arithmetic and logic unit(ALU): The ALU module shown in the figure 3.3b can be useful for performing the computation and branch instructions. We can use the existing comparison functionality used for SLT(set less than) instruction and XOR instruction for checking greater or less than and equality comparison branch instructions respectively.

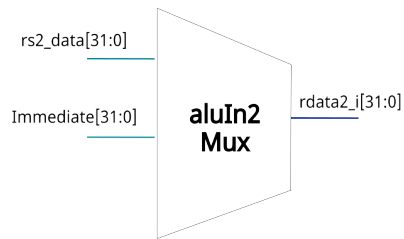
Instruction Memory(InstructionMemory_with_write) module shown in figure 3.1a gives us the instructions based on the Instruction address. I have designed this module suitable for writing its instruction byte-wise so that we can load it with required instructions using a peripheral protocol. **result write back mux** (rslt_Mux) in figure 3.4a is used to select the data, write it to the respective GPR's location based on the control signal coming from the control unit.

The Modules required to design for the types of instructions mentioned in the above table are as follows.

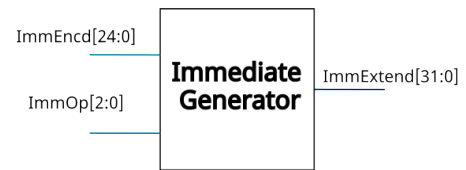


(a) Instruction Memory

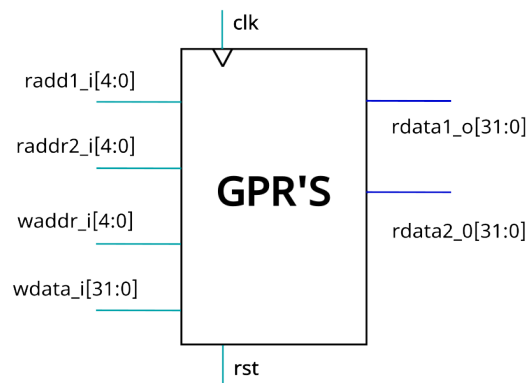
(b) Data Memory



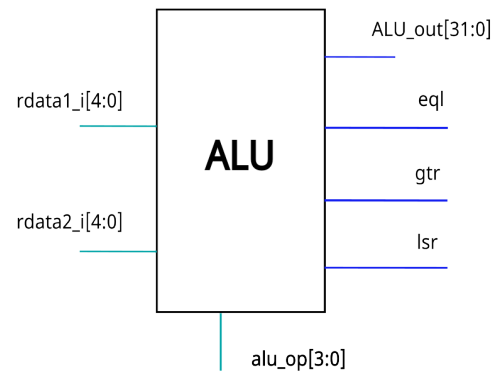
(a) ALU 2nd input mux



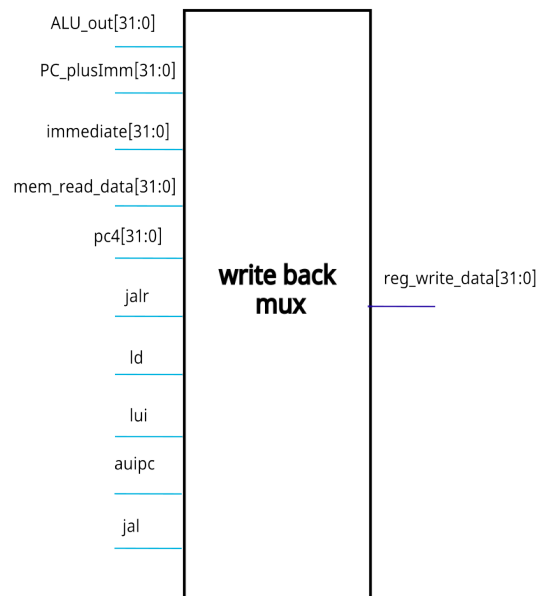
(b) Immediate Generator



(a) General Purpose Registers (GPRs)



(b) Arithmetic Logic Unit (ALU)



(a) Result Write Back Mux

Figure 3.4: Result Write Back Mux

3.2 Single Cycle Architecture

For Single Cycle Design We can realize it in two ways ,In one way we have to use the asynchronous Instruction Memory ,only for an Positive edge triggered GPRs , in a Positive edge triggered Program Counter, with a synchronous Data Memory. In another one , for a positive edge triggered program counter , along with asynchronous instruction memory and synchronous Data Memory , we have to use the negative triggered GPRs.

The overall data Path is shown in the figure 3.5 The Respective Data Paths for each type of instruction are as below. The supportability of the data path for the all types instructions of RV32I mentioned in 2.2 are shown in the diagrams 3.6, 3.7,3.8,3.9,3.10,3.11, 3.12,3.13,3.14,

Data Paths

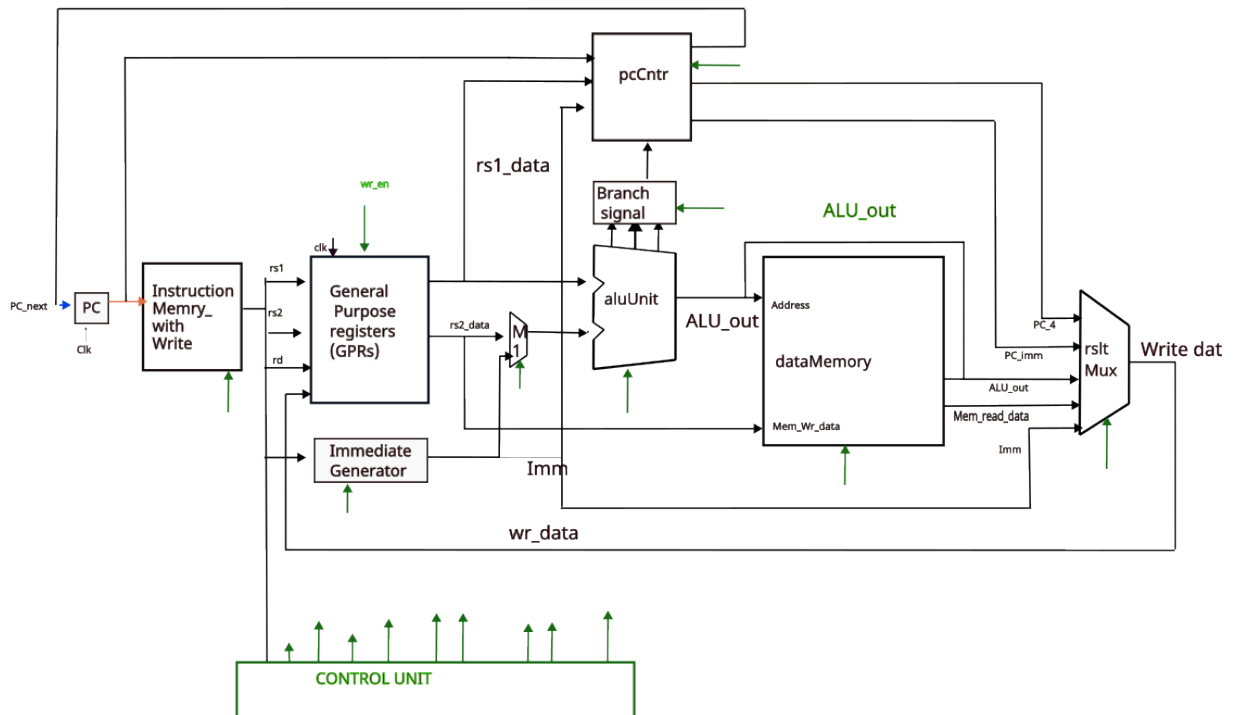


Figure 3.5: RV32I Data Path

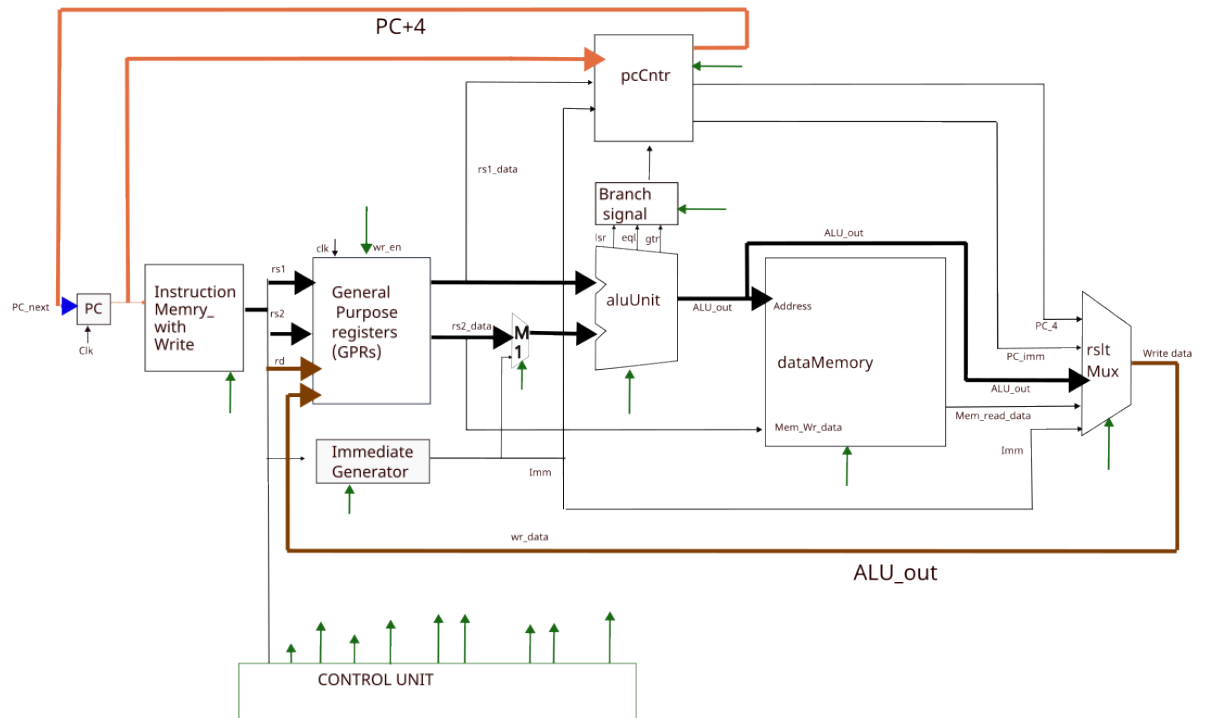


Figure 3.6: R type Instruction data Path

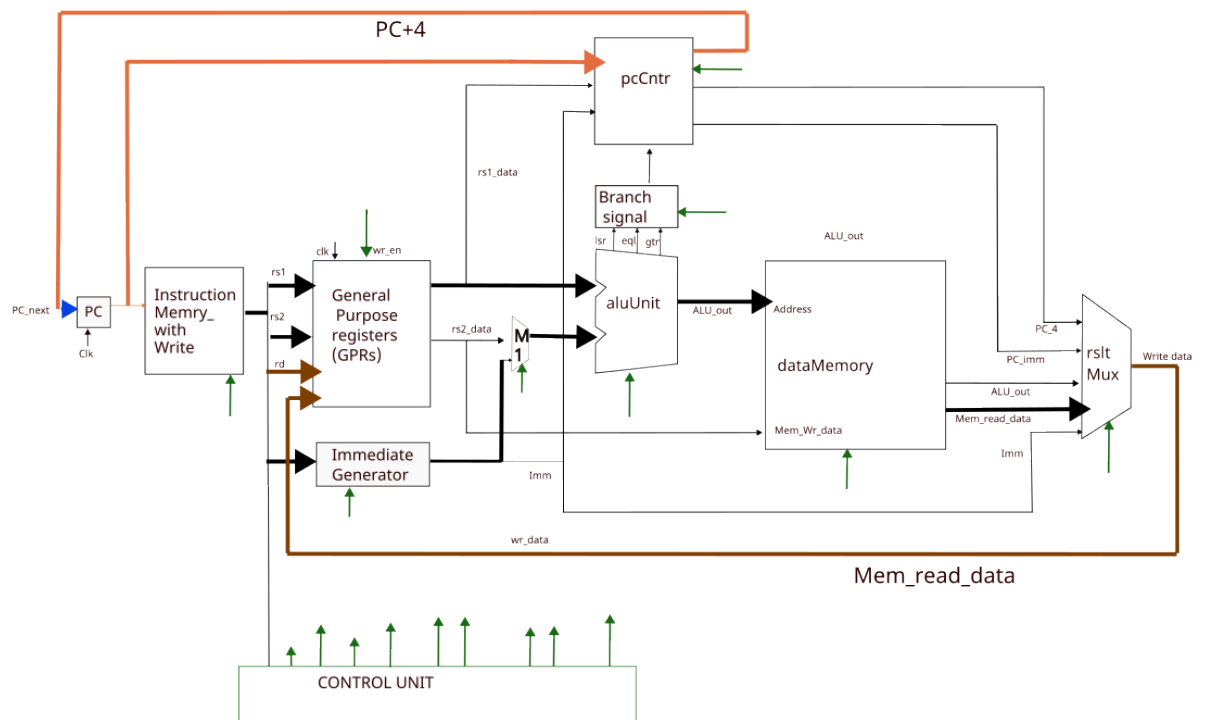


Figure 3.7: L type Instruction data Path

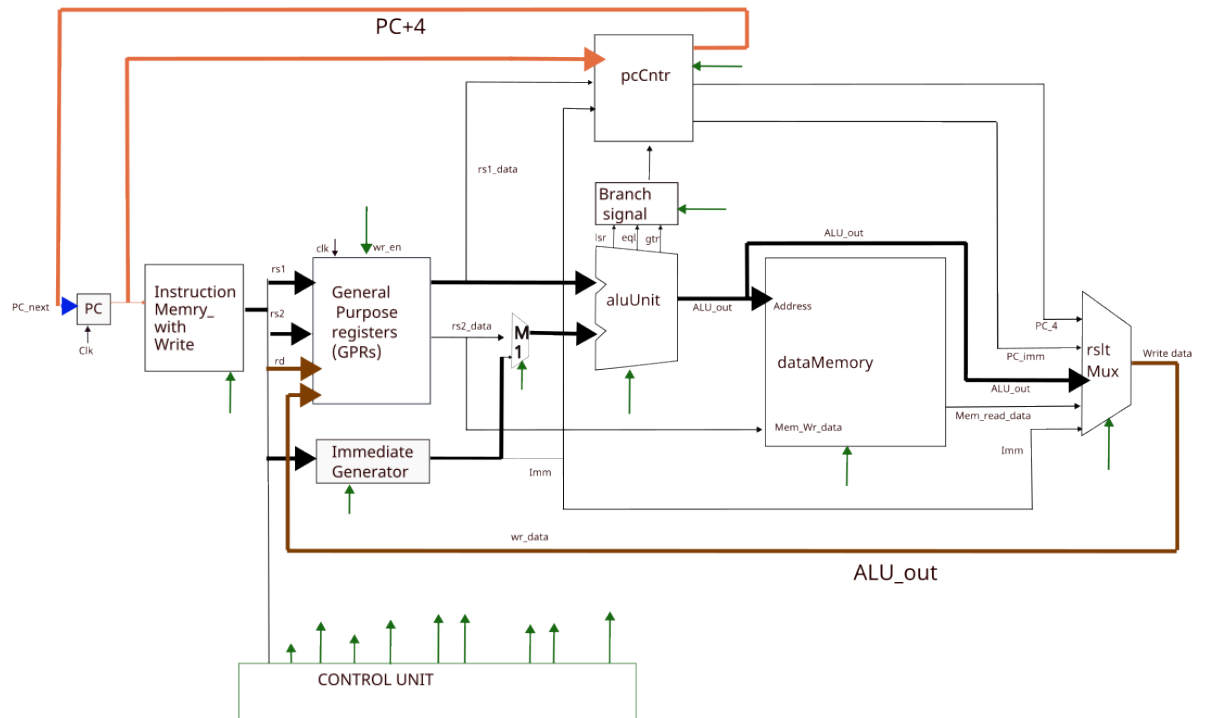


Figure 3.8: RI type Instruction data Path

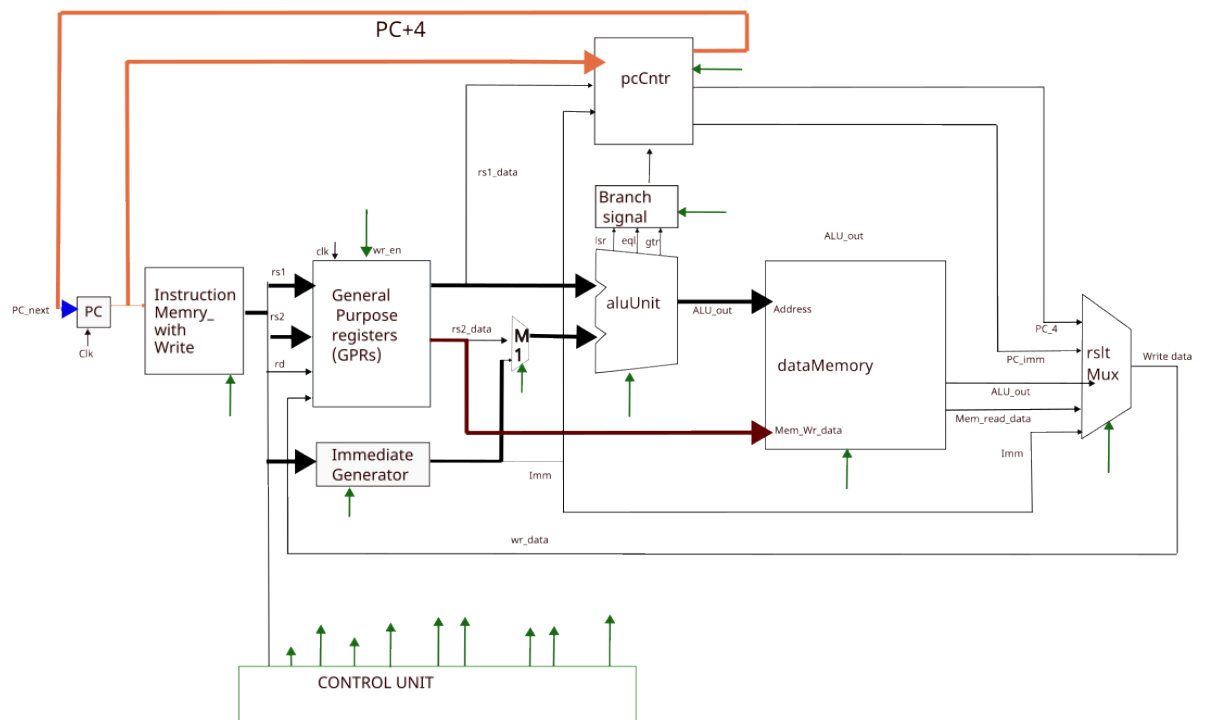


Figure 3.9: S type Instruction data Path

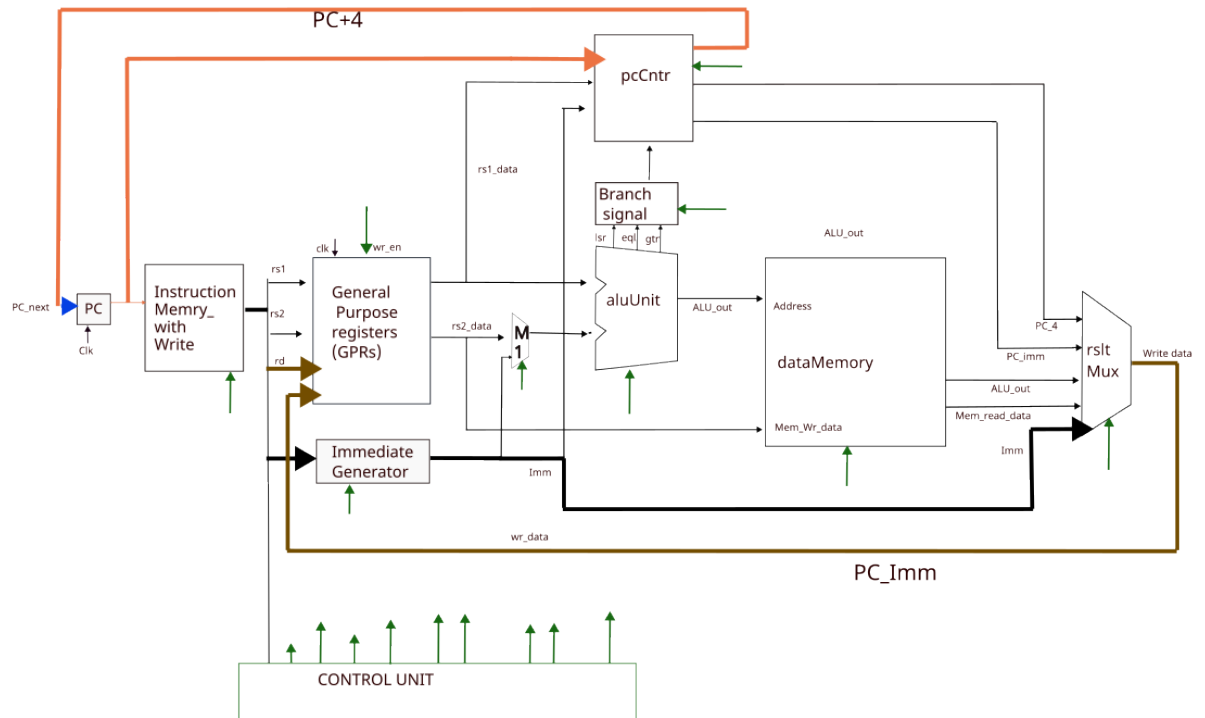


Figure 3.10: LUI type Instruction data Path

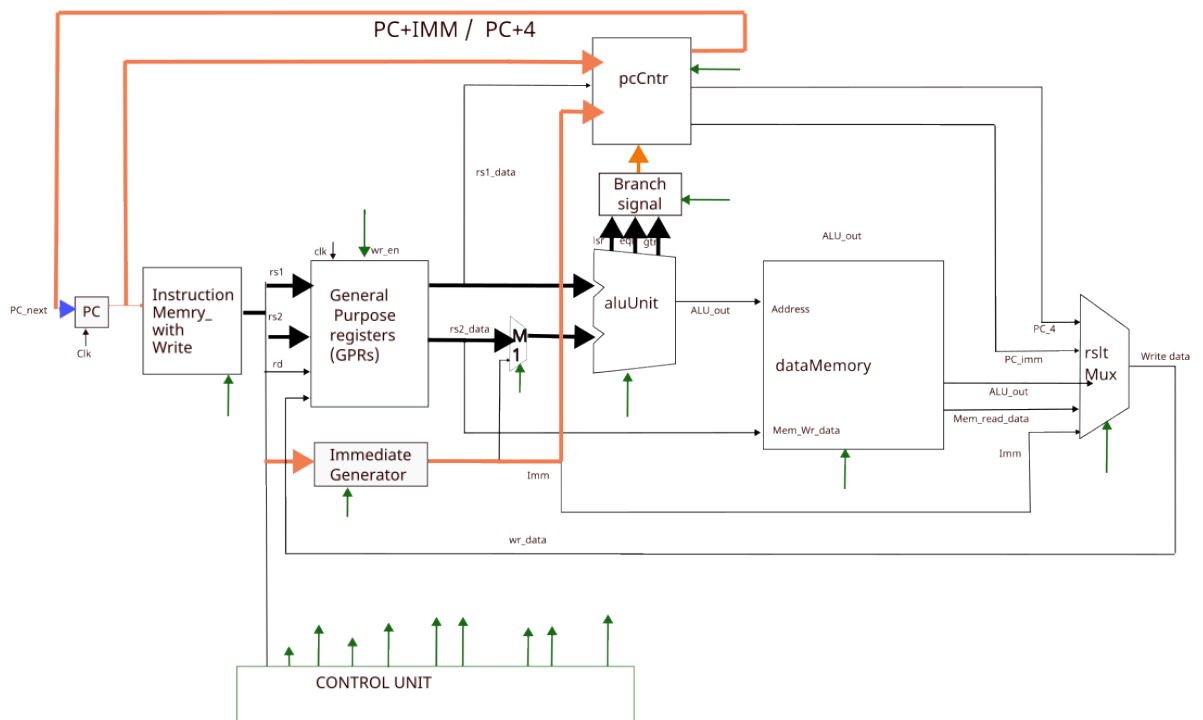


Figure 3.11: BR type Instruction data Path

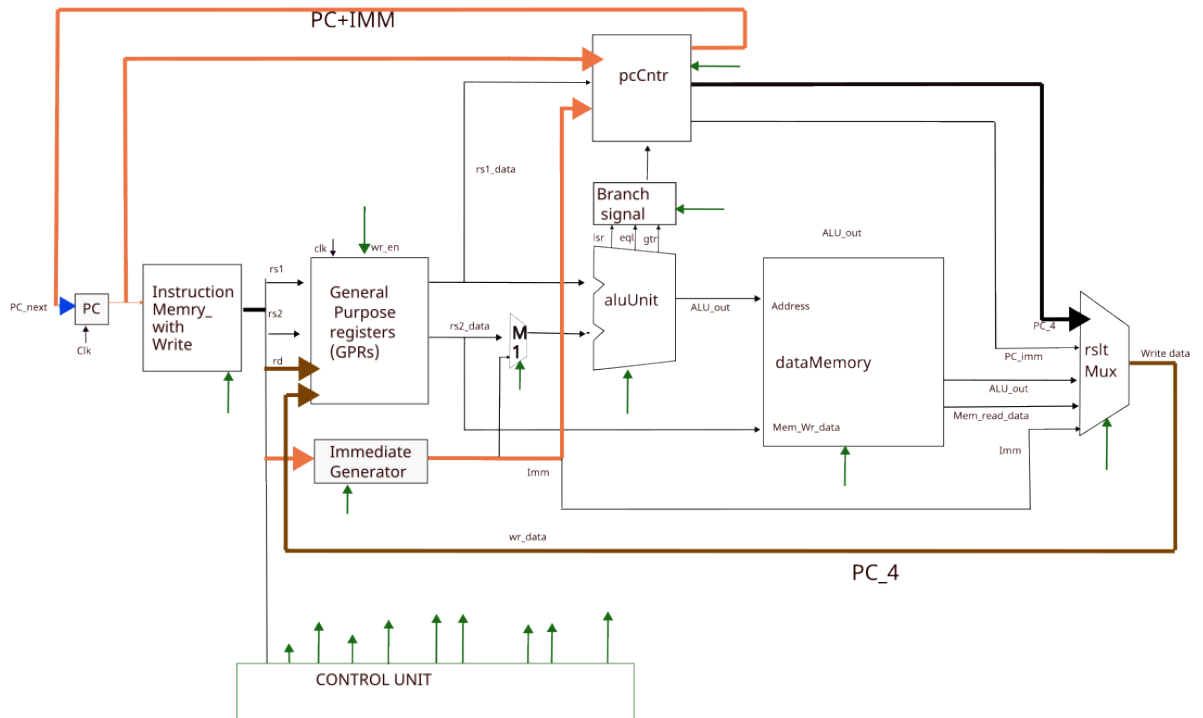


Figure 3.12: JAL type Instruction data Path

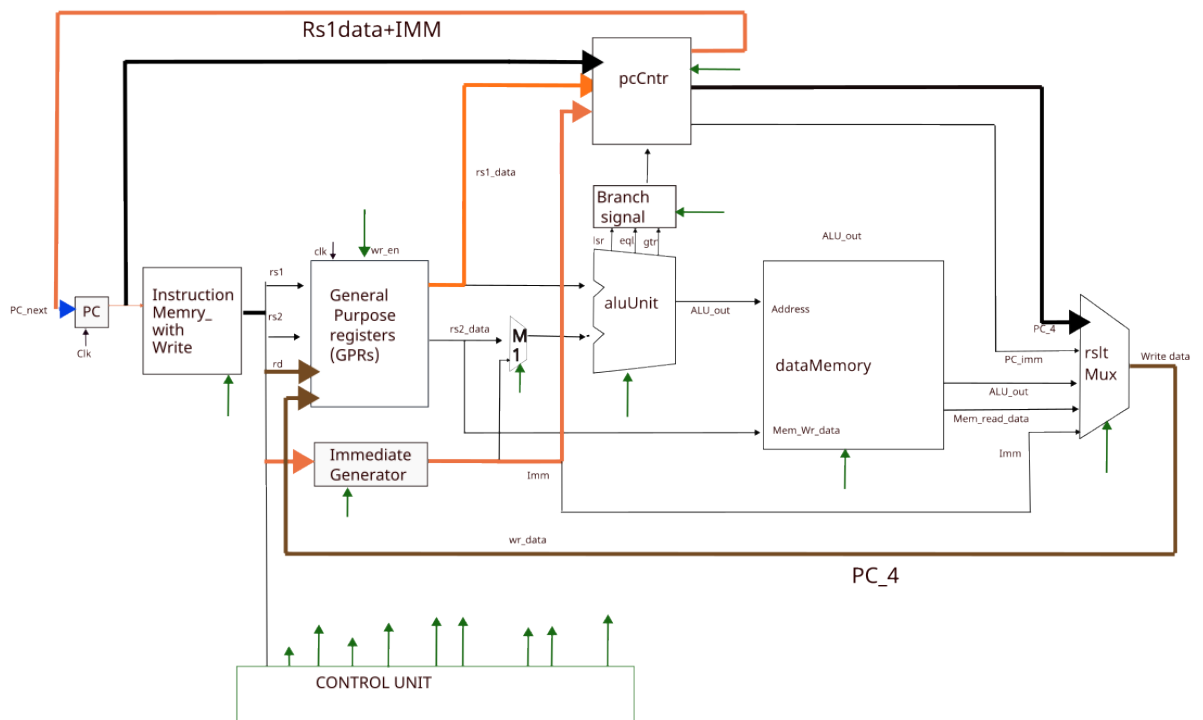


Figure 3.13: JARL type Instruction data Path

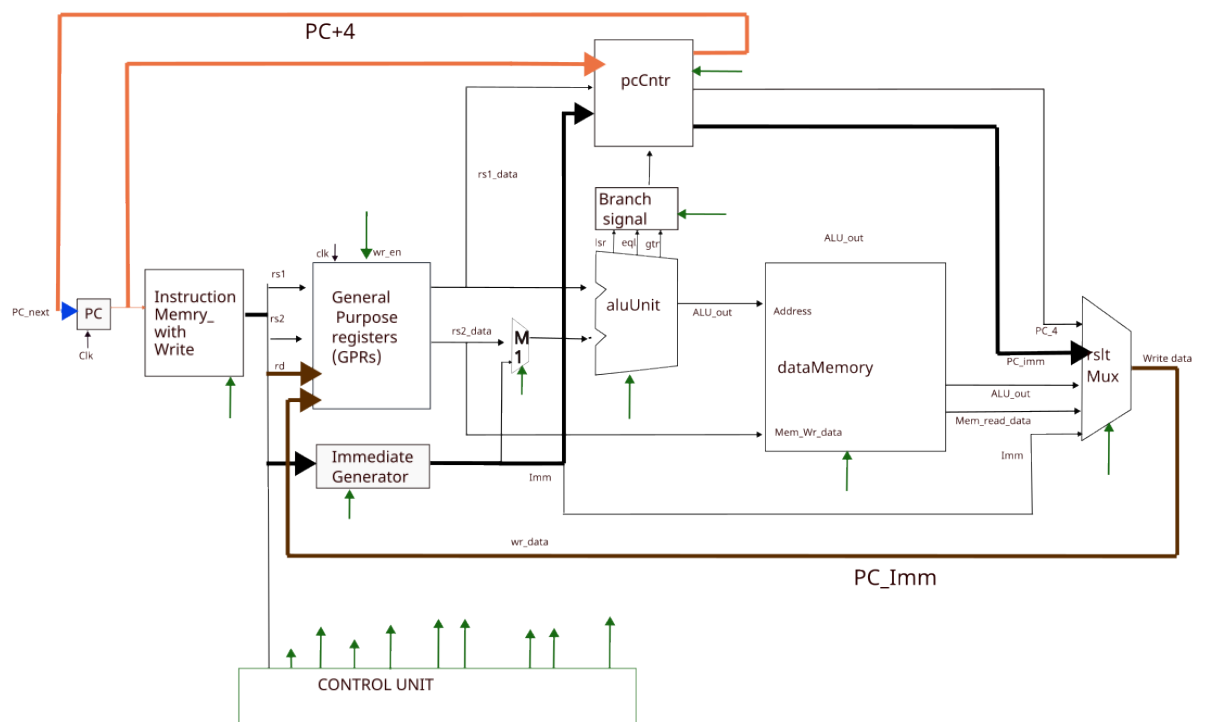


Figure 3.14: AUIPC type Instruction data Path

3.3 Adding M extension to RV32I

RISC-V architecture can incorporate acceleration capabilities, which makes it appropriate for a wide range of applications and sophisticated computing requirements'-V processors like the RV32IM, used as a Digital Signal Processor, that make tasks like AI processing, AI models execution more efficiently compared to the RV32I ISA.

M extension Instructions: Building upon the RV32I base, the RV32M Extension introduces specialized ALU instructions tailored for integer multiplication and division operations. This extension enhances the computational capabilities of RISC-V processors by incorporating instructions such as "MUL" "MULH" , "MULHSU" "MULHU", "DIV" and "REM" These instructions enable efficient handling of complex arithmetic tasks, particularly useful in applications requiring intensive mathematical Computation containing Multiplication and division. The Instructions are listed in Table 3.1. The specific operations of the instructions are mentioned in the Table 3.3 , 3.2

M extension Computation Module :As shown in the Figure 3.15 It is a extra hardware module required to attach it to the existing ALU of base isa, The The multiplication module used is Array Multiplier and Division the module used is based on the repetitive subtracting and comparing method.

The Datapath modification can be done by replacing the base ISA ALU with Modular ALU for this RV32IM. It is the combination of the base ALU and the M extension module along with some muxing at the output port as shown in the Figure 3.16. The multiplication module gives the 64bit output, which is split into MSB results and LSB results, in the same way the division the module also gives the 32-bit Remainder and quotient, so by using muxes, we can select the required result

The extra Module required to generate the required control signals for the extra M extension module is as shown in Figure 3.17. Its respective control signals are listed in tables 3.4 and 3.5 for Multiplication and Divisions respectively.

Instruction	Funct7	rs2	rs1	Funct3	rd	Opcode Extension
MUL	0000001	rs2	rs1	000	rd	0110011
MULH	0000001	rs2	rs1	001	rd	0110011
MULHSU	0000001	rs2	rs1	010	rd	0110011
MULHU	0000001	rs2	rs1	011	rd	0110011
DIV	0000001	rs2	rs1	100	rd	0110011
DIVU	0000001	rs2	rs1	101	rd	0110011
REM	0000001	rs2	rs1	110	rd	0110011
REMU	0000001	rs2	rs1	111	rd	0110011

Table 3.1: RV32M Standard Extension Instructions

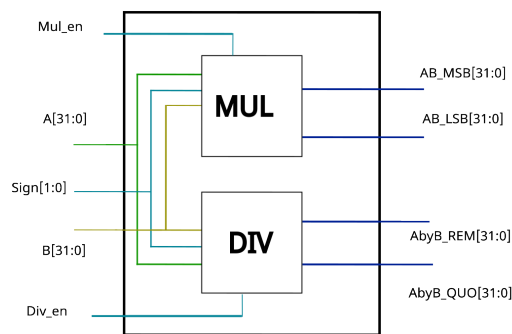


Figure 3.15: M extension Computation Module

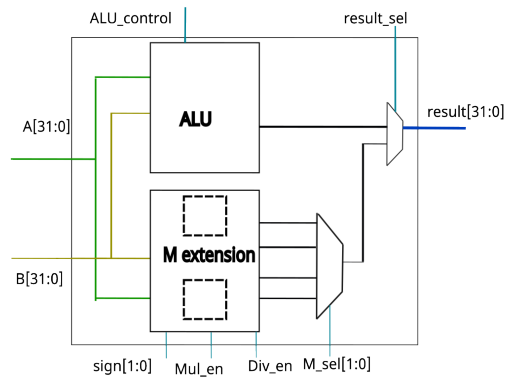


Figure 3.16: ALU Module for RV32IM

The Data Path required for the RV32IM is shown in the below figure [3.18](#).

Instruction	(Funct3)	Rd_data	Data (32-bit)	
			Rs1 (Multiplicand)	Rs2 (Multiplicand)
MUL	000	LSB 32-bit result	Signed	Signed
MULH	001	MSB 32-bit result	Signed	Signed
MULHSU	010	MSB 32-bit result	Signed	Unsigned
MULHU	011	MSB 32-bit result	Unsigned	Unsigned

Table 3.2: Multiplication Instructions operation

Instruction	(Funct3)	Rd	Data (32-bit)	
			Rs1 (Dividend)	Rs2 (Divisor)
DIV	100	Quotient	Signed	Signed
DIVU	101	Quotient	Unsigned	Unsigned
REM	110	Remainder	Signed	Signed
REMU	111	Remainder	Unsigned	Unsigned

Table 3.3: Division Instructions Operation

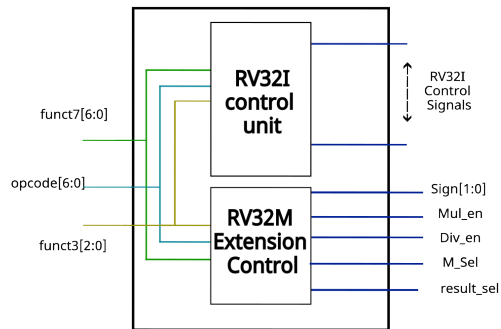


Figure 3.17: M extension Control Module

Instruction (Funct3)	Sign	Mul_en	Div_en	M_sel	result_
MUL (000)	11	1	0	10	1
MULH (001)	11	1	0	11	1
MULHSU (010)	10	1	0	11	1
MULHU (011)	00	1	0	11	1

Table 3.4: Multiplication Instructions Control signals

Instruction (Funct3)	Sign	Mul_en	Div_en	M_sel	result_
DIV (100)	11	0	1	00	1
DIVU (101)	00	0	1	00	1
REM (110)	11	0	1	01	1
REMU (111)	00	0	1	01	1

Table 3.5: Division Instructions Control signals

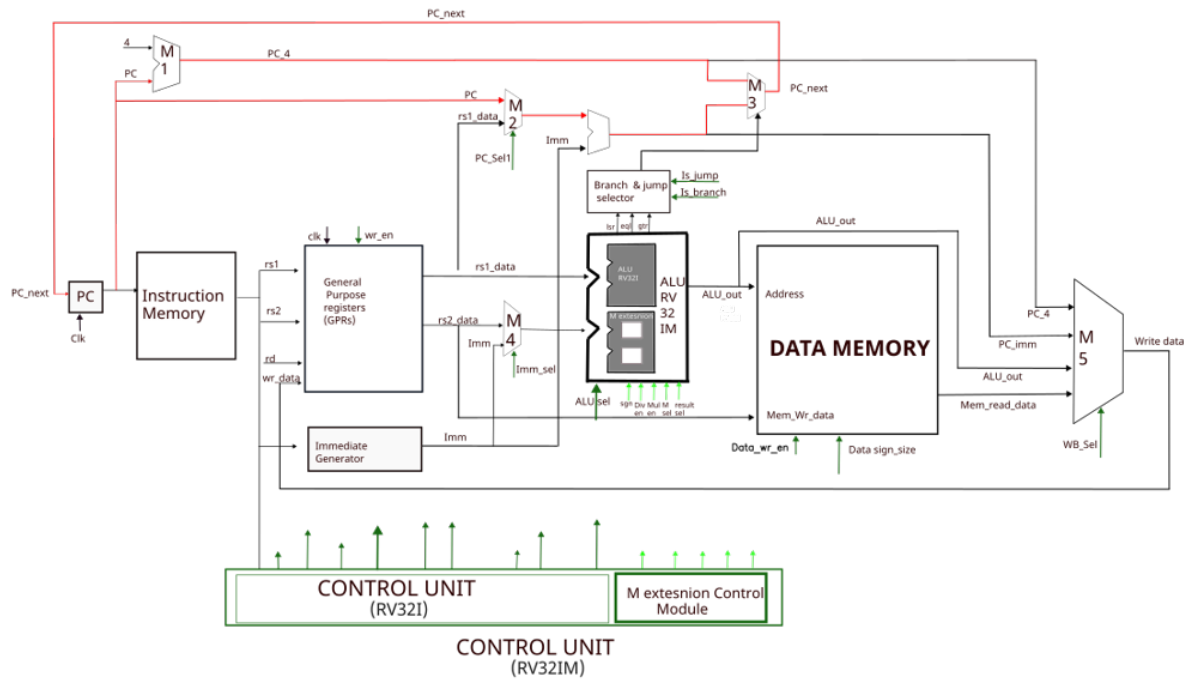


Figure 3.18: rv32im data path

3.4 Pipelining Design OF RV32IM

3.4.1 Designing

The whole process can be seen in five different stages: Instruction Fetch, Instruction Decode, Computation Execution, and Result Memory Write. We need to add a few buffers, such as IF_ID buffer, ID_EX buffer, EX_MEM buffer, and MEM_EX buffer, as shown in the 3.19, between each state to increase the clock frequency of our processor design.

Here, the Branch Detection is happening in the Decoding stage with a cost of a slight increase in delay, In this kind of design there is no need of a Branch predictor

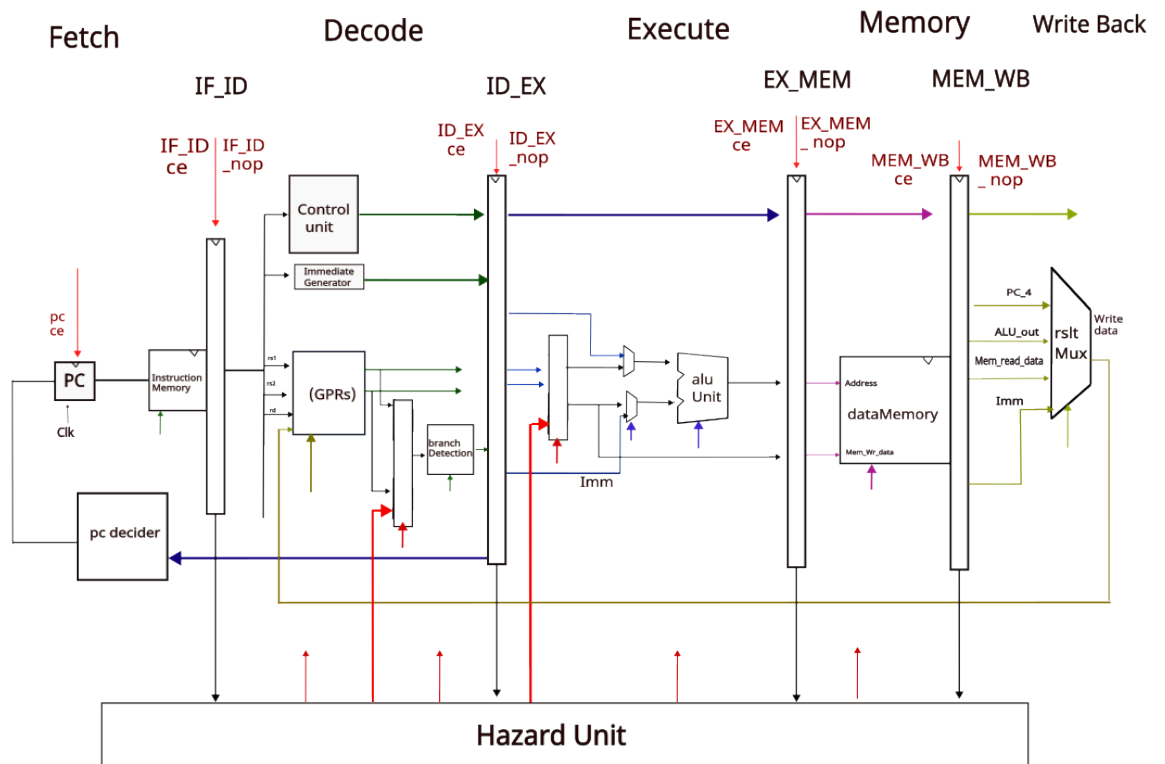


Figure 3.19: Pipelined Microarchitecture Design

To Design Pipelining Design We need to deal with Hazards that occur due to Instruction Dependencies on Data, Hardware and Control .

3.4.2 Hazards

Data Hazards occur when there are Data Dependencies between the instructions. In the figure 3.20 , the outputs of the first instructions and the 2nd instruction are needed to be the inputs of the 3rd instructions , but in pipelining design , the write back happens in the last stage , the 3rd instructions which is supposed to load the required data from the GPRs , could load due to the **RAW**(Read after Write) Hazard .

So one way of resolving this is using a Data forwarder in the execution stage, so that when a later instruction needs data computed by the former instructions then after the computation of the former instructions computation, those required data by the later instruction would be forwarded using a data forwarding unit as shown in the figure. the required control signals for the data forwarding unit would come from the hazard unit

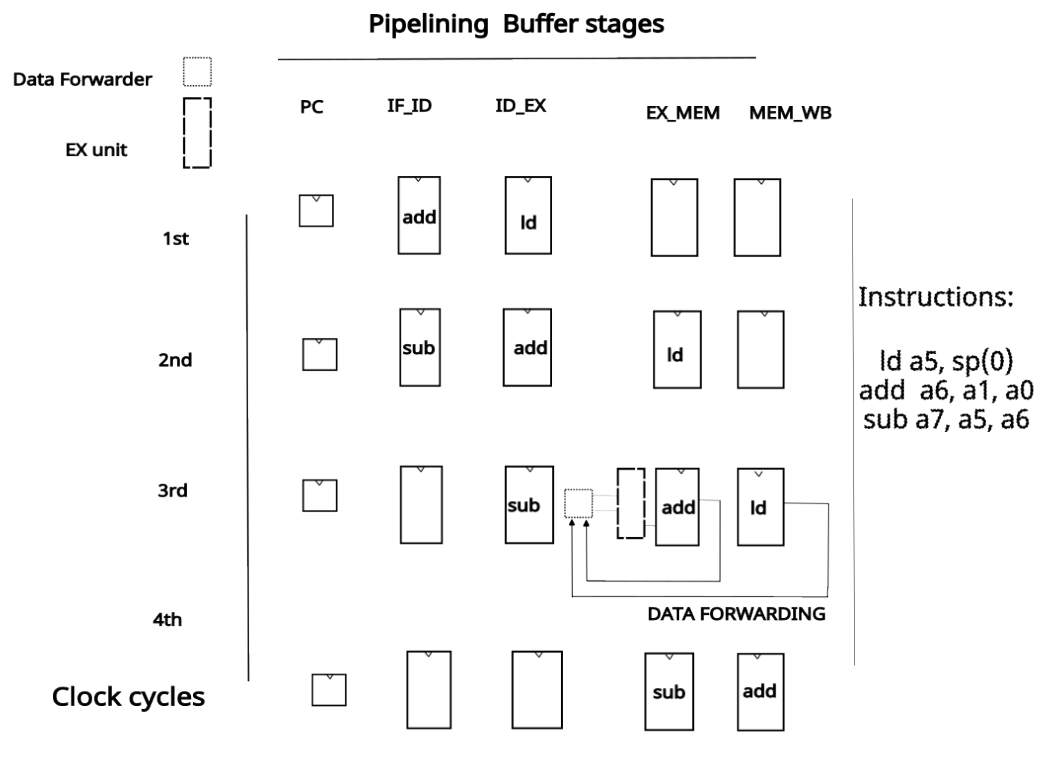


Figure 3.20: Data Hazard Handling

Control Hazard occurs due to the branch and jump instructions, which are used to call instructions of other addresses. In the figure 3.21 if we see in the instruction decode stage, we will know that the type of instructions is jump. at that time there would be the following instruction **in1**, **in2** in the pipeline, so in the next clock cycle, the instruction

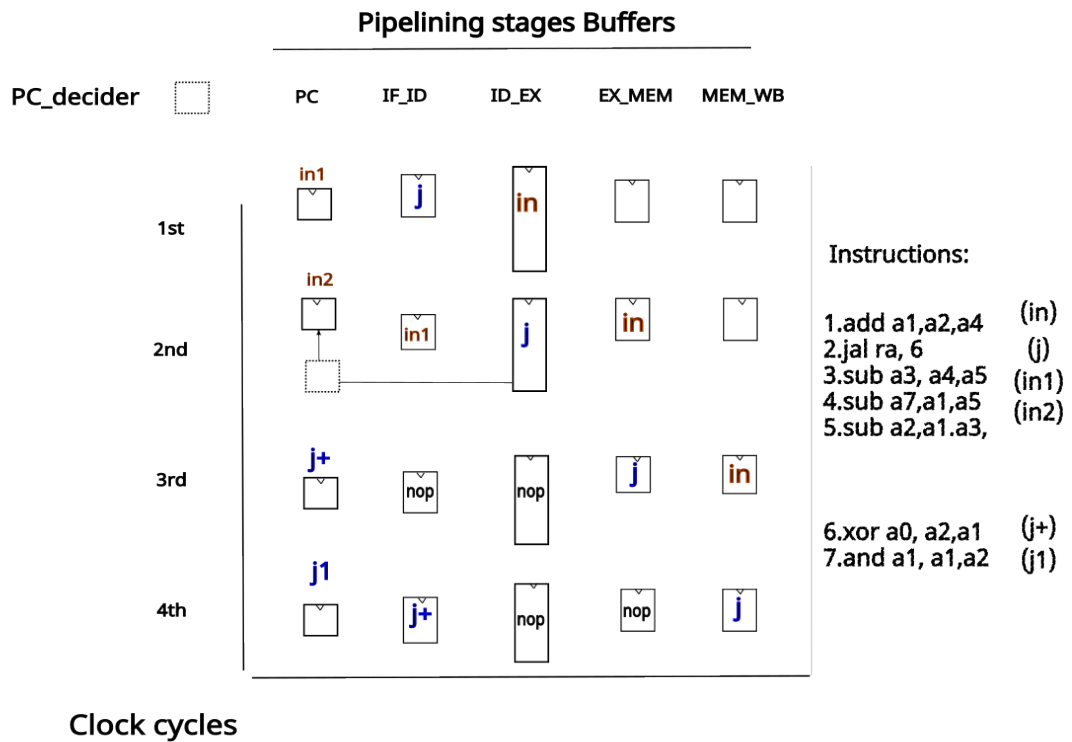


Figure 3.21: Control Hazard Handling

data in the buffers IF_ID, ID_EX would be set to zero as **nop**(no operation) and the instruction that the jump instructions is directing to would be stored in the program counter. meanwhile, the **in** instruction would pass through the pipeline normally

CHAPTER 4

VERIFICATION METHODOLOGY

I have used the assembly codes having different test cases for individual instructions from this github source [riscv_tests](#). then converted them to hexadecimal data using the toolchain and placed those instructions into the processor instruction memory, in vivado, then verified the Processor functionality to support all instructions.

4.1 Softwares used

[RISC-V_gnu_toolchain](#) It is used to convert the C/C++ code to the RISC-V instructions according to our required ISA, along with extension Instructions if we require them .

Xilinx Vivado :The whole Designing of single cycle and pipelined processor is designed and simulated, synthesized using the Vivado 2022.2 version

4.2 Simulation Based Verification

Below are the screenshots of waveforms of the processor signals when control and data hazards occurs. The waveform [4.2](#) is a simulation of the data-dependent consecutive instructions highlighted in the figure [4.4](#), when they occur the Forwarding unit forwards the data from the pipelined available data to the execution unit .The forwarding unit forwards a)Data from the Memory available in the Write back stage, b)ALU computation data available in the Memory stage c) computed ALU output data writing back to the GPR's in the Write back stage. In the same way, the waveform [3.21](#) is the simulation of the few instructions, which has jump instructions in it as highlighted in the figure [4.3](#). When a jump instruction is detected in the execution stage, then the execution stage and the decoding stage will not operate (nop-all data is reset to zero) in the next clock cycle. After another clock cycle, the instructions at the jump address come to the decoding stage.

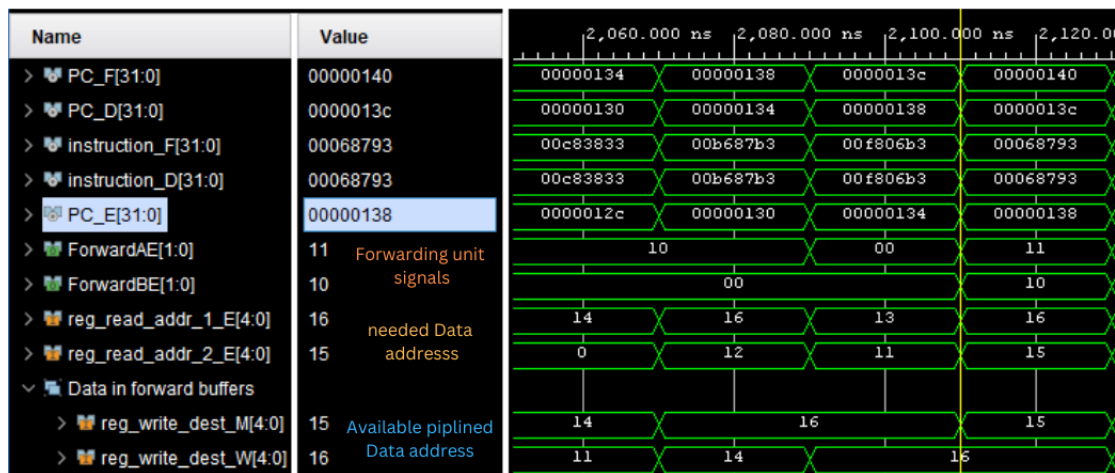


Figure 4.1: Data Hazards

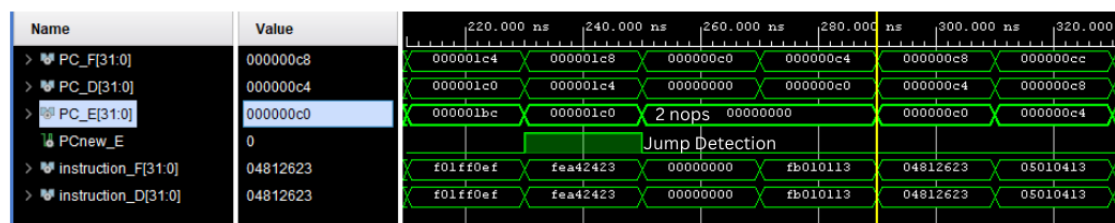


Figure 4.2: Control Hazards

```

000001ac <main>:
1ac:   addi sp,sp,-32
1b0:   sw ra,28(sp)
1b4:   sw s0,24(sp)
1b8:   addi s0,sp,32
1bc:   addi a0,zero,25
1c0:   jal ra,c0 <fibonacci_sequence>
1c4:   sw a0,-24(s0)
1c8:   sw a1,-20(s0)

00000c0 <fibonacci_sequence>:
c0:   addi sp,sp,-80
c4:   sw s0,76(sp)
c8:   addi s0,sp,80
cc:   sw a0,-68(s0)
d0:   addi a5,zero,0

```

Figure 4.3: Instructions Causing Control Hazard

```

12c:   addi a6,a4,0
130:   sltu a6,a6,a2
134:   add a5,a3,a1
138:   add a3,a6,a5
13c:   addi a5,a3,0
140:   sw a4,-40(s0)
144:   sw a5,-36(s0)
148:   lw a4,-32(s0)

```

Figure 4.4: Instructions Causing Data Hazards

4.2.1 Arithmetic operation programme

The code in figure 4.5 , has multiplication ,Division, addition , subtraction operations on the given data, The final output of the Computation is 70 , which can seen in the the simulation

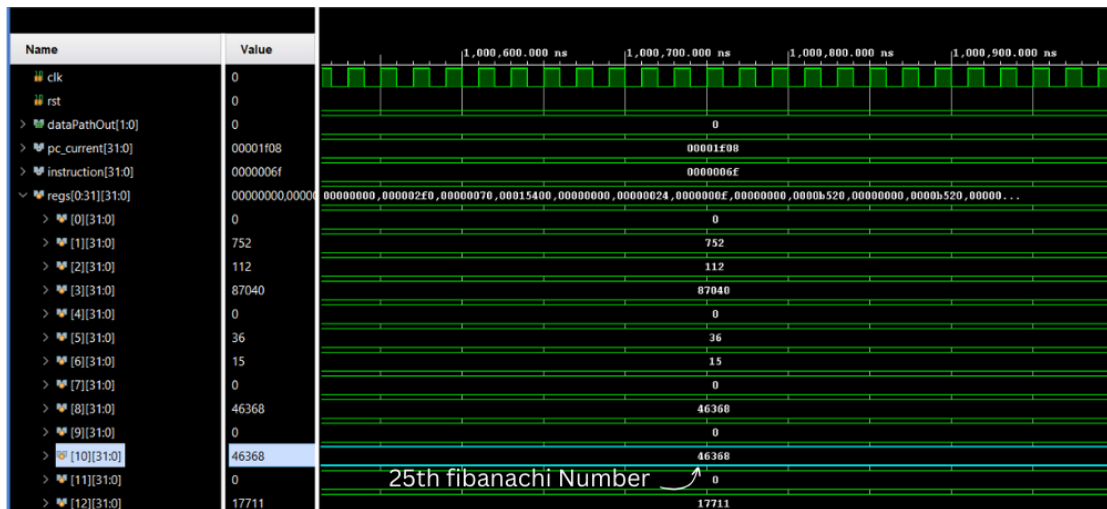


Figure 4.7: 25th Fibonacci number calculation

4.3 FPGA based verification

A c++ program written in such a way that it would return the result , using the risc-v gnu toolchain is converted into a text file containing the assembly and hexadecimal instructions data . Now, a Python script is used to extract the hexadecimal instructions data and write them in a new text file. Now using a UART transmission protocol Python, the hexadecimal data is sent to the FPGA implemented process. Now after enabling the start push button in the processor to enable it to run the program instructions loaded in it. After a After executing all the instructions on the board, We can send the data memory contents to our laptop while running a Python script for UART receiving ptotocal in our laptop. Now, we can see the contents of the data memory using a text file.

Based on our program returning value, we can check whether that computed data is there or not in the data memory contents.

VERIFICATION METHODOLOGY:

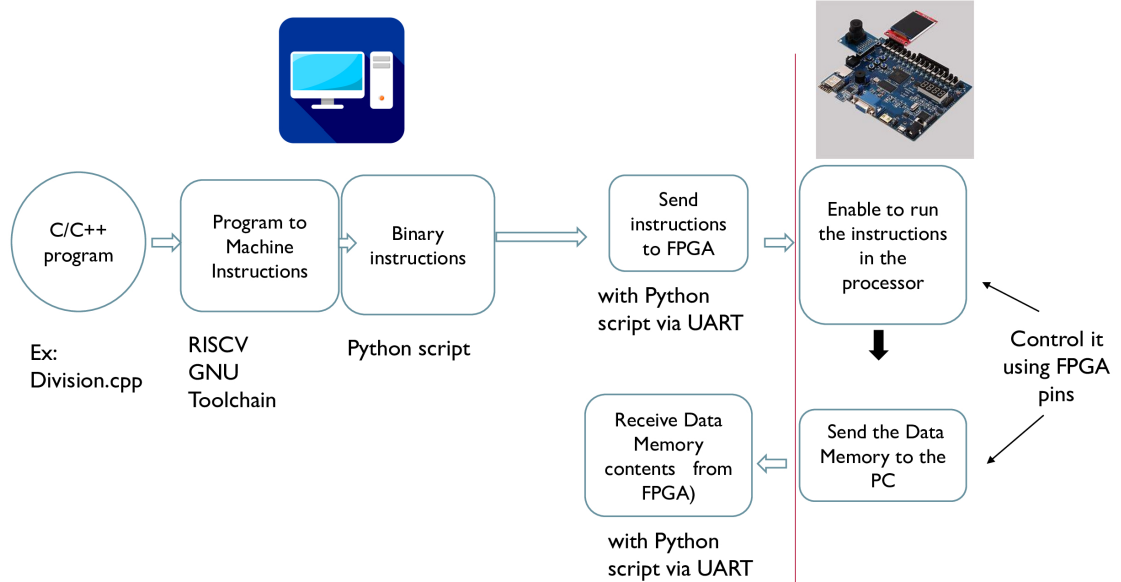


Figure 4.8: FPGA Verification Flow

CHAPTER 5

RESULTS AND DISCUSSION

I have successfully Designed and run the programs on the RV32IM-based pipelined RISC-V processor. Below are the results of in terms of delay

Singel Cycle	Pipelined
90.082ns	59.525ns

Table 5.1: Delay results of RV32IM single cycle vs Piplined Designs

The reason for nearly only half reduction in the delay even in the five stage pipelined design is the Major delay contributor is ALU unit with MUltiplicaiton and DIvision Support whose delay is nearly equal to the pipelined delay which we got, so due to the larger delay contributor in execution stage , we can only see 2 fold delay reduction , but not 5 fold

Processsor	LUT	LUTRAM	BRAM
Single Cycle	8336	5168	0
Pipelined	5431	1855	17

Table 5.2: FPGA resource utlization

The reason for the No BRAM utilization in the single cycle, is due to lack of asynchronous reading memory modules in the design, where BRAM would only be utilized for synchronous memory read and writes. Meanwhile n pipelined design , the synchronous read can be used

CHAPTER 6

SUMMARY AND CONCLUSION

This project can be improved/continued in the following aspects

1. Converting this processor into the SOC and implementing OS on it
2. With help of I/O interfacing programs such as keyboard, Display , we can make a setup to run a computation program by giving inputs to via keyboard and by seeing outputs on Display connected to the FPGA-implemented processor
3. Improving the Processor speed, by adding more pipelined stages in the MULTplication and DIVision computation modules as well.

REFERENCES

1. **S. L. H. M. Harris**, *Digital_design_and_computer_Architecture_RISCV_editio*. ELSEVIER, .
2. **NEO32_Processor** (). Neorv32_processor. <https://github.com/stnolting/neorv32>.
3. **PICORV32_Processor** (). Picorv32_processor. <https://github.com/YosysHQ/picorv32>.
4. **r1** (). <https://drive.google.com/drive/u/0/home>.
5. **RISC-V_gnu_toolchain** (). Risc-v_gnu_toolchain. <https://github.com/riscv-collab/riscv-gnu-toolchain>.
6. **riscv_tests** (). riscv tests for individual instructions. <https://github.com/AngeloJacob/RISC-V/tree/main/test/extra>.
7. **A. Waterman** (2017). The risc-v instruction set manual: Volume i user-level isa version 2.2. https://drive.google.com/file/d/1s0lZxUZaa7eV_00_WsZzaurFLLww7ou5/view.
8. **A. Waterman** and **K. Asanovic** (2019). Unprivileged specification version 20191213. https://drive.google.com/file/d/1s0lZxUZaa7eV_00_WsZzaurFLLww7ou5/view.

APPENDIX A

Processor ISA Types

A.0.1 Background Intro

Generally, a processor performs a specific computation on the required data. Any computation we require is communicated to the processor through INSTRUCTIONS, which the processor supports. The Only language a processor speaks is in terms of instructions. The only language that a processor can understand is its instruction language.

Any programming language that we have to translate(compiled exactly)into the instructions of that processor on the PC.

- a) Stack-based ISA: Data comes from Memory and forms a data stack. Like a LIFO (Last In, First Out) way, elements added last are the ones removed first. The top elements of the stack are used as operands for arithmetic or logical operations. Here, with a type of instruction (Push A, Push B), the data that comes from Memory is stored temporarily as a data stack. Then, computation is performed with an instruction (Add) on the data stack, and finally, the output is sent to the memory with an instruction (Pop C).

Example:Java Virtual Machines work on stack based ISA

- a) Accumulator-based ISA: An accumulator-based ISA involves arithmetic and logic operations using the accumulator as one of the operands and storing the result back in the accumulator.

Here, the computation happens between the data from direct memory and data in the accumulator with an instruction (Add B) after computation, the result would occur in the accumulator itself, and then finally, the result in the accumulator could be stored back in memory with an instruction(Store C).

Example: The Intel 8085 microcontroller is on accumulator-based ISA.

- a) Register-to-memory ISA: In a register-to-memory ISA, data is transferred between registers and memory. Instructions in this type of ISA typically involve loading data from memory into registers or storing data from registers into memory.

For example, For a computation mentioned in the Figure, operand one is temporarily stored in registers initially with an instruction(Load R1, A), and then the computation is performed with an instruction(Add R1, B) on operand one from registers and operand two from Memory. The output is stored in a register only, and then, finally, the result in the register could be stored back in memory with an instruction (Store R1, C).

Example: x86 architecture.is based on reg to memory ISA

- a) Register-to-register(Load/Store) ISA: In a register-to-register ISA, operations are performed directly between registers without involving memory. Initially, instructions load the required data from the memory to the registers and manipulate data within registers, then store it back with instructions. In the figure, the initial instructions(Load R1, A, Load R2, B) loaded the required data from memory to the registers, and then computation was performed on the data in the respective registers with an instruction(Add R3, R2, R1), then the output is stored in the register R3. Finally, the result is stored in the memory via an instruction(Store R3, C).

Example: RISC-V,MIPS, and ARM architectures are based on reg-to-reg ISA

Each type of ISA has its own characteristics and influences the design and behaviour of the processor it governs. The selection of ISA type affects the processor architecture's complexity, speed, and capabilities.

APPENDIX B

Data Path Modules Description and functionalities

Port	Description
clk	Clock signal
InstrWrAdd	Instruction write address
InstrWrData	Instruction write data (byte)
InstrWrEn	Instruction write enable
pc	Program counter
instruction	Instruction output
start	Start control signal
InstrWrEn (Control)	Instruction write enable (control signal)

Table B.1: Instruction Memory Ports Description

Start	Operation
1	instruction = Instruction_Memory[PC];
0	instruction = 32'd0

Table B.2: Instruction Memory Asynchronous Reading for a single cycle

Start	clk	Operation
1	posedge	instruction <= Instruction_Memory[PC];
0	posedge	instruction <= 32'd0

Table B.3: Instruction Memory Synchronous Reading for Multi-cycle or Pipelined Design

InstrWrEn	clk	Operation
1	posedge	Instruction_Memory[InstrWrAdd] <= InstrWrData;
0	posedge	X

Table B.4: Instruction Memory Synchronous Writing

"<=" is a synchronous operation with the clock, which takes a positive edge triggered clock cycle for the operation to perform

"=" Asynchronous operation , independent of the clock

Signal	Description
clk	Clock signal
mem_access_addr[31:0]	Memory access address
mem_write_data[31:0]	Data to be written to memory
mem_read_data[31:0]	Data read from memory
mem_write	Memory write control signal
mem_read	Memory read control signal
byte_half_word	Byte/half-word selection control signal
full_word	Full-word selection control signal
byteU	Unsigned byte selection control signal
half_wordU	Unsigned half-word selection control signal

Table B.5: Data Memory Ports

byte	half_word	full_word	mem_access_addr[1:0]	data_to_be_stored
1	X	X	00	=mem_write_data[7:0]
1	X	X	01	=mem_write_data[15:8]
1	X	X	10	=mem_write_data[23:16]
1	X	X	11	=mem_write_data[31:24]
0	1	X	X0	=mem_write_data[15:0]
0	1	X	X1	=mem_write_data[31:16]
0	0	X	XX	=mem_write_data[31:0]

Table B.6: Data Memory data to be stored control signals

mem write	clk	Operation
1	posedge	dataMemory[mem_access_addr[31:2]] <= data_to_be_stored;
0	posedge	X

Table B.7: Data Memory Synchronous Writing

Data_Reading_signals = {byte, half_word, byteU, half_wordU}		
Data_Reading_signals	mem_access_addr[1:0]	reading_data
X X X X	XX	No read operation
1 X X X	00	{memory[ram_addr][7:0]}
1 X X X	01	{memory[ram_addr][15:8]}
1 X X X	10	{memory[ram_addr][23:16]}
1 X X X	11	{memory[ram_addr][31:24]}
0 1 X X	X0	{memory[ram_addr][15:0]}
0 1 X X	X1	{memory[ram_addr][31:16]}
0 0 1 X	00	{24'd0, memory[ram_addr][7:0]}
0 0 X 1	00	{16'd0, memory[ram_addr][15:0]}
0 0 0 X	01	{24'd0, memory[ram_addr][15:8]}
0 0 X 0	01	{16'd0, memory[ram_addr][31:16]}
0 0 X X	10	{24'd0, memory[ram_addr][23:16]}
0 0 X X	11	{24'd0, memory[ram_addr][31:24]}

Table B.8: DataMemory reads operation control signals

mem read	clk	Operation
1	posedge	mem_read_data <= reading_data;
0	posedge	mem_read_data <= 32'd0

Table B.9: Data Memory Synchronous Reading

clk	gprsWren	Read	Write
x	0	rdata1_o= GPRs_[raddr1_i], rdata2_o= GPRs_[raddr2_i]	x
posedge	1	rdata1_o= GPRs_[raddr1_i], rdata2_o= GPRs_[raddr2_i]	GPRs[waddr_i] <= wdata_i

Table B.10: GPR's synchronous write & asynchronous read

ImmOp	Operation
000	I-type
001	S-type
010	B-type
011	J-type
100	U-type
Default	Undefined

Table B.11: ImmOp and Operation

Operation	ImmExtD Output
I-type	{{20{ImmEncd[24]}}, ImmEncd[24:13]}
S-type	{{20{ImmEncd[24]}}, ImmEncd[24:18], ImmEncd[4:0]}
B-type	{{20{ImmEncd[24]}}, ImmEncd[0], ImmEncd[23:18], ImmEncd[4:1], 1'b0}
J-type	{{12{ImmEncd[24]}}, ImmEncd[12:5], ImmEncd[13], ImmEncd[23:14], 1'b0}
U-type	{ImmEncd[24:5], 12'b000000000000}
Undefined	32'd0

Table B.12: Immediate Generator Functionality

alu_op	Operation	ALU_out	lsr	gtr	eql
0000	ADD	$in_1 + in_2$	0	0	0
1000	SUB	$in_1 - in_2$	0	0	0
0010	SLT	1 if $in_1 < in_2$	ALU_out[0]	!(ALU_out[0])	0
0011	SLTU	1 if unsigned($in_1 < in_2$)	ALU_out[0]	!(ALU_out[0])	0
0111	AND	$in_1 \& in_2$	0	0	0
0110	OR	$in_1 in_2$	0	0	0
0100	XOR	$in_1 \wedge in_2$	0	0	!(ALU_out)
0001	SLL	$in_1 \ll in_2[4:0]$	0	0	0
0101	SRL	$in_1 \gg in_2[4:0]$	0	0	0
1101	SRA	$in_1 \ggg in_2[4:0]$	0	0	0
Default	No operation	0	0	0	0

Table B.13: ALU unit control signals

ld	jal	jalr	auipc	lui	Output (reg_write_data)
0	0	0	0	0	ALU_out
1	0	0	0	0	mem_read_data
0	1	0	0	0	pc4
0	0	1	0	0	pc4
0	0	0	1	0	PC_plusImm
0	0	0	0	1	immediate

Table B.14: write back :rsltMux control signals

APPENDIX C

Code snippets

Below is the Python code to extract the hexadecimal instructions form the text file generated form toolchain:

Listing C.1: Python code to extract data

```
import re
import os

def extract_data(input_file_path):
    try:
        input_file_name, input_file_extension =
            os.path.splitext(input_file_path)
        output_file_path =
            f"{input_file_name}_extracted_machine_n_Assembly.txt"
        machine_code_output_file_path =
            f"{input_file_name}_machine.txt"

        with open(input_file_path, 'r') as file:
            data = file.read()
            hex_data =
                re.findall(r'\b[0-9a-fA-F]+:\s+([0-9a-fA-F]+\s+',
                    data)

        with open(output_file_path, 'w') as file:
            file.write('\n'.join(hex_data))

        with open(machine_code_output_file_path, 'w') as file:
            file.write('\n'.join(hex_data))

    return True, output_file_path,
        machine_code_output_file_path # Return True to
```

```

        indicate the operation was successful and the paths
        of the output files
    except Exception as e:
        print(f"An error occurred: {e}")
        return False, None, None # Return False to indicate that
            an error occurred and no output files were created

# Taking input file path as input
input_path = input("Enter the input file path: ")
extraction_result, output_file, machine_code_output_file =
    extract_data(input_path)

if extraction_result:
    print(f"Filtered data has been saved to the output files:
        {output_file} and {machine_code_output_file}")
else:
    print("An error occurred while processing the file.")

```

Below is the Python code snippet for receiving data form FPGA using UART protocol:

Listing C.2: Python code for serial communication with FPGA

```

import serial
import binascii

# Replace "COM14" with the correct COM port where your FPGA is
    connected
serial_data = serial.Serial("COM14", 9600)

# Open a new text file for writing
with open("Data_Mem111.txt", "w") as file:
    consecutive_data = b"" # Initialize an empty byte string
        for consecutive data

    while True:

```

```

dat = serial_data.read(1) # Read 1 byte of data from FPGA

# Convert the received data to hexadecimal
data_hex = dat.hex()

# Append the data to the consecutive_data byte string
consecutive_data += binascii.unhexlify(data_hex)

# If we have received 4 consecutive bytes, print them in
    little-endian order
if len(consecutive_data) == 4:
    little_endian_data = consecutive_data[::-1] # Reverse
        the order
    print("Little-endian data:", little_endian_data.hex())

# Write the little-endian data to the text file
file.write(little_endian_data.hex() + "\n")

# Reset the consecutive_data for the next set of 4
    bytes
consecutive_data = b""

# Close the serial connection (this line will not be reached
    as the loop runs indefinitely)
serial_data.close()

```

Below is the Python code snippet for send instructions from laptop to FPGA using UART protocol:

Listing C.3: Python code to send data via UART

```

import serial
import time

def send_data_via_uart(input_file_path):
    try:

```

```

    # Open the input file and read data
    with open(input_file_path, 'r') as file:
        data = file.read()

    # Open serial connection
    ser = serial.Serial('COM14', 9600, timeout=1) # Fixed
        COM port

    # Send data through UART
    ser.write(data.encode())
    print("Data sent successfully.")

    # Close serial connection
    ser.close()

    return True # Return True to indicate successful
        transmission
except Exception as e:
    print(f"An error occurred: {e}")
    return False # Return False to indicate transmission
        failure

# Input file path
input_file_path = input("Enter the input file path: ")

# Send data through UART
send_result = send_data_via_uart(input_file_path)

if send_result:
    print("Data sent via UART successfully.")
else:
    print("Failed to send data via UART.")

```
